

Learning Systems: Trees, Ensembles and Networks

Tanujit Chakraborty

Senior Research Fellow, Indian Statistical Institute, Kolkata.

Founder, STAT&ML Lab.

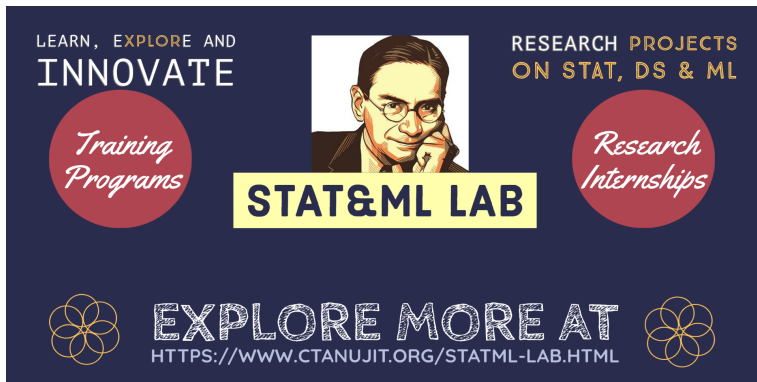
Email: tanujitisi@gmail.com

July 30, 2020



- **STAT&ML Lab** is a non-profit organisation to bring young minds into research projects on Statistics & Machine Learning.
- We aim to provide training and research projects on the key universal tool of modern science & technology “**Statistics**” and cutting-edge tool like “**Machine Learning**” in the promotion of human welfare”
- **Primary goal** of this lab is to promote research in Statistics in India and throughout the world.
- **Secondary aim** of the STAT&ML Lab is to provide training programs on Data Science, Statistics and Machine Learning to young motivated students.
- We look for highly motivated and research-minded M.Sc / M.Tech students who are willing to solve “**Application-driven problems**” from various interdisciplinary fields.

- To know more about STAT&ML Lab: <https://www.ctanujit.org/statml-lab.html>
- Ongoing Projects at our Lab: <https://www.ctanujit.org/research-projects.html>
- Courses at our Lab: <https://www.ctanujit.org/courses.html>



A promotional banner for the STAT&ML LAB. The banner has a dark blue background. At the top left, the text "LEARN, EXPLORE AND INNOVATE" is written in white, with "EXPLORE" in orange. Below this is a red circle containing the text "Training Programs" in white cursive. In the center is a portrait of a man with glasses, resting his chin on his hand. Below the portrait is a yellow rectangle with the text "STAT&ML LAB" in bold black letters. To the right of the portrait is another red circle containing the text "Research Internships" in white cursive. At the top right, the text "RESEARCH PROJECTS ON STAT, DS & ML" is written in white, with "RESEARCH PROJECTS" in orange. At the bottom, the text "EXPLORE MORE AT" is written in large, white, textured letters, followed by the URL "HTTPS://WWW.CTANUJIT.ORG/STATML-LAB.HTML" in white. On either side of the URL is a logo consisting of five interlocking circles.

LEARN, EXPLORE AND
INNOVATE

Training Programs

STAT&ML LAB

RESEARCH PROJECTS
ON STAT, DS & ML

Research Internships

EXPLORE MORE AT
[HTTPS://WWW.CTANUJIT.ORG/STATML-LAB.HTML](https://www.ctanujit.org/statml-lab.html)

Faculty Members of STAT&ML Lab



Dr. Munmun Biswas



Dr. Debolina Ghatak



Dr. Suchismita Das



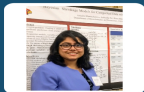
Mr. Raju Maiti



Mr. Swarup Chattopadhyay



Mr. Bikash Santra



Dr. Arinjita Bhattacharyya



Ms. Aparajita Khan



Mr. Indrajit Ghosh



Mr. Sourav Maji



Mr. Sourav Nandi



Mr. Srijit Mukherjee

Student Members of STAT&ML Lab



Zubia Mansoor



Kaumil Trivedi



Gauri Kamat



Sayak Banerjee



Shramana Bhattacharya



Tejasvi Arora

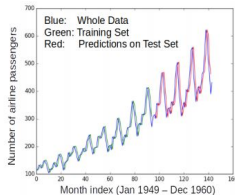


Tirna Mahajan



Poulami Chowdhury

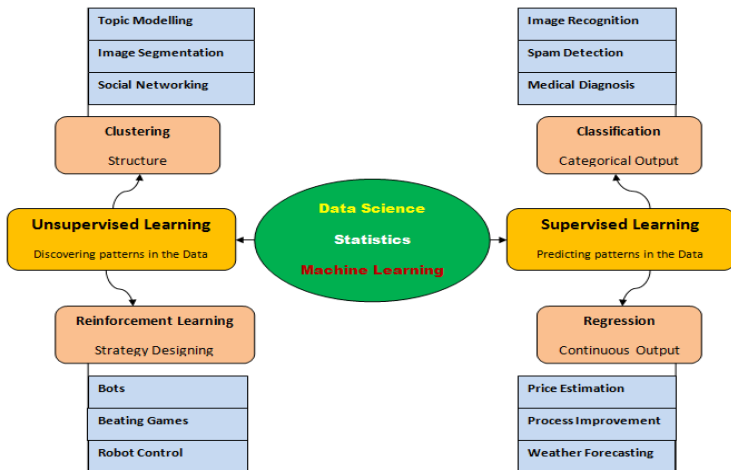
- Designing algorithms that **ingest data** and **learn a model** of the data.
- The learned model can be used to
 - ① Detect **patterns/structures/themes/trends** etc. in the data
 - ② Make **predictions** about future data and make decisions



- Modern learning systems are highly **“data-driven”**.
- Optimize a performance criterion using example data or **past experience**.

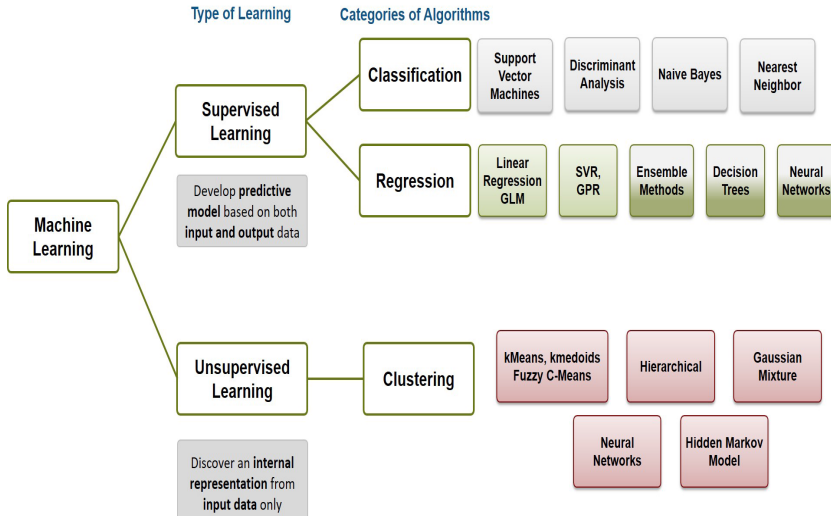
Taxonomy for Learning Systems

Learning systems provides systems the ability to automatically learn.



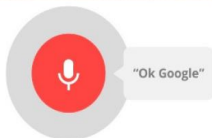
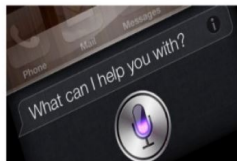
Machine Learning

A computer program is said to learn from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E . - A definition by Tom Mitchell.



Real-world Applications of Machine Learning

Broadly applicable in many domains (e.g., internet, robotics, healthcare and biology, computer vision, NLP, databases, computer systems, finance, etc.).



Predictive Policing



Online Fraud Detection

Development of Learning Systems

- Linear Regression (Galton, 1875).
- Linear Discriminant Analysis (R.A. Fisher, 1936).
- Logistic Regression (Berkson, JASA, 1944).
- k-Nearest Neighbor (Fix and Hodges, 1951).
- Parzen's Density Estimation (E Parzen, AMS, 1962)
- Classification and Regression Tree (Breiman et al., 1984).
- Artificial Neural Network (Rumelhart et al., 1985).
- Perceptron Trees (Paul Utgoff, 1989, Connection Science).
- MARS (Friedman, 1991, Annals of Statistics).
- SVM (Cortes and Vapnik, Machine learning, 1995)
- Random forest (Breiman, 2001).
- Deep Convolutional Neural Nets (Krizhevsky, Sutskever, Hinton, NIPS 2012).
- Generative Adversarial Nets (Ian Goodfellow et al., NIPS 2014).
- Deep Learning (LeCun, Bengio, Hinton, Nature 2015).
- Bayesian Deep Neural Network (Yarin Gal, Islam, Zoubin Ghahramani, ICML 2017).

k-Nearest Neighbors

- kNN does not build model from the training data.
- To classify a test instance d , define k -neighborhood P as k nearest neighbors of d .
- Count number n of training instances in P that belong to class c_j .
- Estimate $\Pr(c_j|d)$ as n/k .
- No training is needed. Classification time is linear in training set size for each test case. (Cover & Hart, IEEE IT 1967).

Example: $k=6$ (6NN)

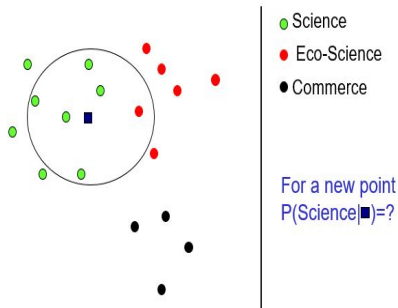


Fig: An example of k-NN

Definition (the kNN classifier)

The kNN classifier: decide $f(X)$ by majority vote among the labels of the training points in the k -nearest neighborhood of X .

- k is usually chosen empirically via a validation set or cross-validation by trying a range of k values.
- Distance function play a crucial role, but depends on applications.
- kNN can deal with complex and arbitrary decision boundaries.
- kNN is slow at the classification time.
- kNN does not produce an understandable model.
- To avoid ties: chooses k odd.

Indoor or Outdoor?



Decision Trees

- Decision tree is defined by a hierarchy of rules (in form of a tree).
- Rules from the internal nodes of the tree are called root nodes
- Each rule (internal node) tests the value of some feature.
- Labeled training data is used to construct the Decision tree. The tree need not to be always a binary tree.
- CART (Breiman et al., 1984), RF (Breiman, 2001), BART (Chipman et al., 2010).

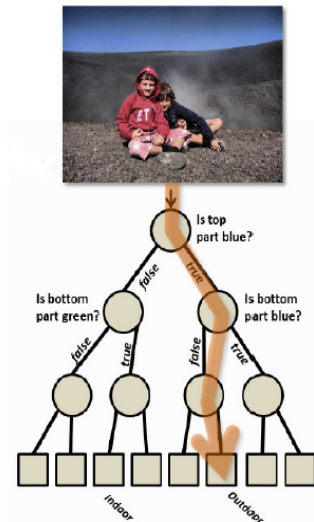


Fig: An example of a Classification Tree

Pros & Cons of Decision Trees

- CART is a greedy divide-and-conquer algorithm. Trees are constructed in a top-down recursive manner based on selected attributes.
- Attributes are selected on the basis of an impurity function (e.g., IG for Classification & MSE for Regression).
- **Pros:** Built-in feature selection mechanism, Comprehensible, easy to design, easy to implement, good for structural learning.
- **Cons:** But may become large for complex problems, too many rules loose interpretability, risk of over-fitting, sticking to local minima.

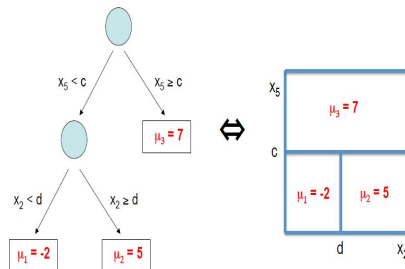


Fig: An example of a Regression Tree

Ensemble Methods

- Problem: Single classifiers have the drawbacks of sticking to local minimum or over-fitting the data set, etc.
- Complex learning problems can often be solved by effectively combining multiple weak model via ensemble learning methods.
- Examples: Bagging, Boosting, Stacking and Voting Method.
- Caution: But ensembles don't always improve accuracy of the model but tends to increase the error of each individual base classifier.

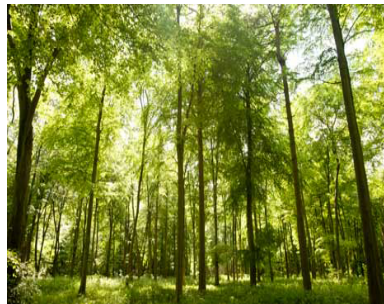


Fig: "Alone we can do so little; together we can do so much". -

Helen Keller

- Bagging stands for Bootstrap Aggregation.
- Takes original data set D with N training examples.
- Creates M copies $\{\hat{D}_m\}_{m=1}^M$
 - Each $\hat{D}_m$ is generated from D by sampling with replacement.
 - Each data set $\hat{D}_m$ has the same number of examples as in data set D (though not all are unique).
 - These data sets are reasonably different from each other.
- Train models $\{h_1, \dots, h_M\}$ using $\{\hat{D}_1, \dots, \hat{D}_M\}$, respectively.
- Use an averaged model $h = \frac{1}{M} \sum_{m=1}^M h_m$ as the final model.
- Useful for models with high variance and noisy data.
- My observation: Bagging is computationally more efficient and performs more accurate than boosting.

- An ensemble of decision tree classifiers, can handle high-dimensional settings.
- Uses bagging on features (each DT will use a random set of features).
 - Given a total of D features, each DT uses $\sqrt{D}$ randomly chosen features.
 - Randomly chosen features make the different trees uncorrelated.
- All DTs usually have the same depth.
- Each DT will split the training data differently at the leaves.
- Prediction for a test example votes on/averages predictions from all the DTs.



Developments of Neural Nets

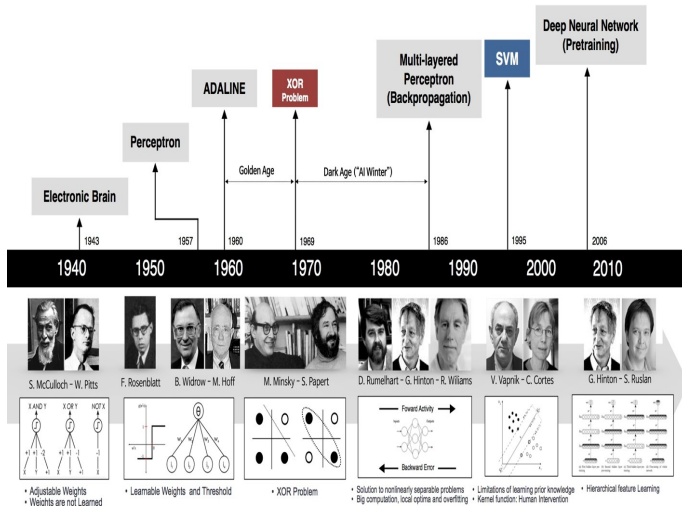
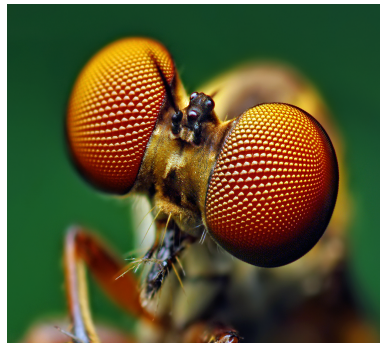


Fig: Developments of Neural Network Models

- The idea of neural networks began as a model of how neurons in the brain function, termed 'connectionism' and used connected circuits to simulate intelligent behaviour.
- In 1943, portrayed with a simple electrical circuit by neurophysiologist Warren McCulloch and mathematician Walter Pitts.
- Donald Hebb took the idea further by proposing that neural pathways strengthen over each successive use, especially between neurons that tend to fire at the same time.

First Perceptron

- Around 1950's, Frank Rosenblatt, a psychologist at Cornell, was working on understanding the comparatively simpler decision systems present in the eye of a fly, which underlie and determine its flee response.
- In an attempt to understand and quantify this process, he proposed the idea of a Perceptron in 1958, calling it Mark I Perceptron.
- It was a system with a simple input output relationship, modeled on a McCulloch-Pitts neuron to explain the complex decision processes in a brain using a linear threshold gate.



Perceptron Machine

- A McCulloch-Pitts neuron takes in inputs, takes a weighted sum and returns 0 if the result is below threshold and 1 otherwise.
- The Mark I Perceptron machine was the first implementation of the perceptron algorithm.
- Limitations:
 - 1 Can't solve non linearly separable problems... (by design).
 - 2 Algorithm not robust to non linear separability!
- Solution: Artificial Neural Networks, e.g., Feedforward Neural Net, Radial Basis Function Network, Bayesian Neural Net).

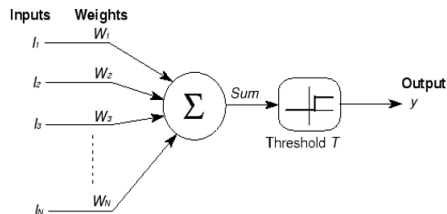
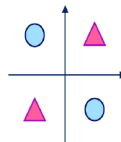


Fig: Most common representation of perceptron



- ANN is composed of several perceptron-like units arranged in multiple layers.
- Consists of an input layer, one or more hidden layer, and an output layer.
- Nodes in the hidden layers compute a nonlinear transform of the inputs.
- Also called a Feedforward Neural Network (since there is no backward connections between layers, viz., no loops).
- Note: All nodes between layers are assumed connected with each other.

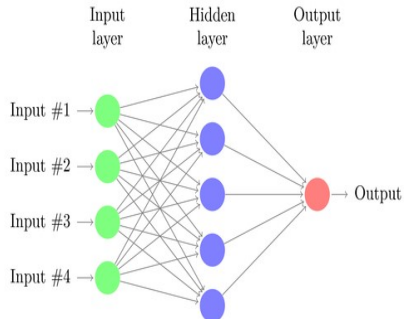
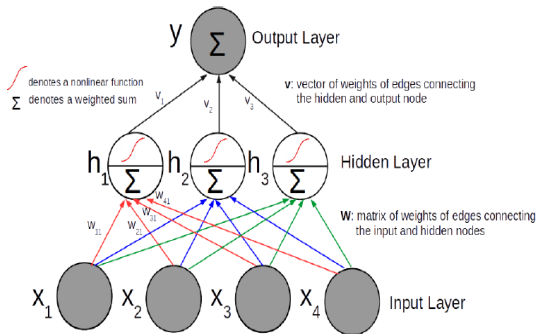


Fig: FFNN Model with one hidden layer with 5 hidden units

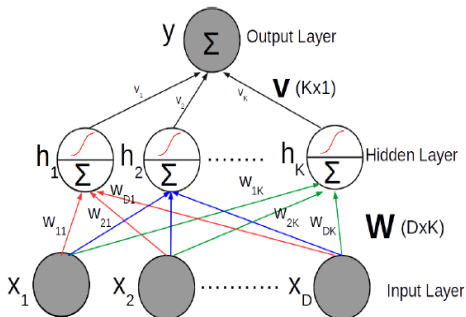
A Simple Feedforward Neural Net

- Below: FFNN with 4 inputs, one hidden layer with 3 nodes, and 1 output.



- Each hidden node computes a nonlinear transformation of its incoming inputs: Weighted linear combination followed by a nonlinear activation function.
- Output y is a weighted combination of the preceding layer's hidden nodes (followed by another transform if y isn't real valued, e.g., binary/multiclass label).

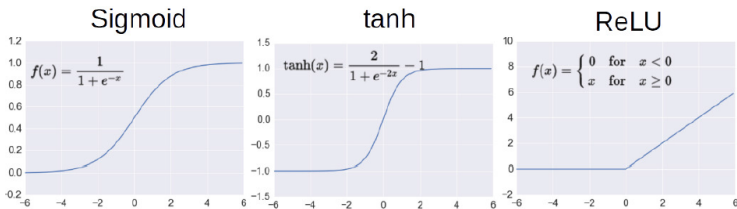
A Simple Feedforward Neural Net



- For an FFNN with D inputs $\mathbf{x} = [x_1, \dots, x_D]$, a single hidden layer with K hidden nodes $\mathbf{h} = [h_1, \dots, h_K]$, and a scalar-valued output node y , each hidden node's value is computed as: $h_k = f(\mathbf{w}_k^T \mathbf{x}) = f\left(\sum_{d=1}^D w_{dk} x_d\right)$ and $\mathbf{h} = f(\mathbf{W}^T \mathbf{x})$.
- Output $y = \mathbf{V}^T \mathbf{h}$, where $\mathbf{V} = [v_1, \dots, v_K] \in \mathbb{R}^K$, $\mathbf{W} = [\mathbf{w}_1, \dots, \mathbf{w}_K] \in \mathbb{R}^{D \times K}$, and f is the nonlinear activation function.

Nonlinear Activation Functions

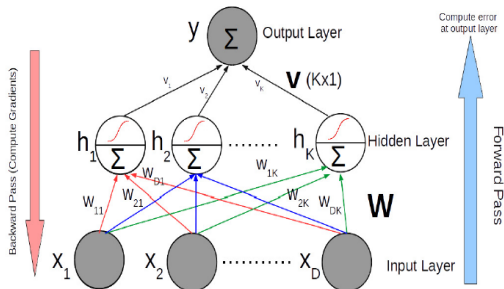
- Some popular choices for the nonlinear activation functions:



- Sigmoid saturates and can kill gradients. Also not “zero-centered”.
- tanh also saturates but is zero-centered (thus preferred over sigmoid).
- Rectified Linear Unit (ReLU): $f(x) = \max(0; x)$ is currently the most popular (also cheap to compute).
- Learn the parameters by minimizing some loss function.
- Backpropagation** (gradient descent + chain rule for derivatives) is commonly used to do this efficiently (Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986)).

Backpropagation

- Basically consists of a forward pass and a backward pass:



- Forward pass computes the errors using the current parameters.
- Backward pass computes the gradients and updates the parameters, starting from the parameters at the top layer and then moving backwards.
- Also good at reusing previous computations (updates of parameters at any layer depends on parameters at the layer above).

Remember: Artificial Neural Networks

- **Universal Approximation Theorem (Hornik, 1989):** A one hidden layer FFNN with sufficiently large number of hidden nodes can approximate any function.
- **Caution:** This result is only in terms of theoretical feasibility. Learning the model can be very difficult in practice (e.g., due to optimization difficulties).
- In Deep Neural Networks, hidden layer can automatically extract features from data. (Hinton 2006, Nature)
- Stochastic gradient descent backpropagation is most popular method for weight optimization.

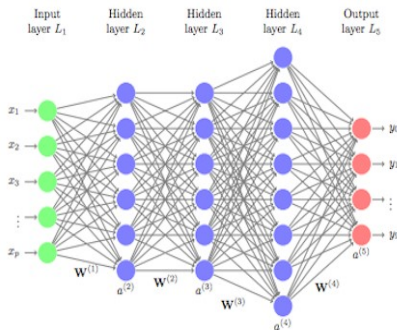
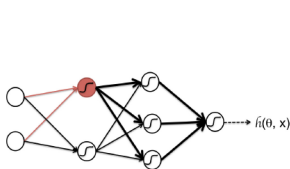


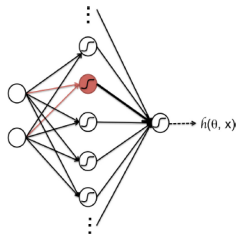
Fig: An example of ANN model with 3 hidden layers

Deep Vs. Shallow Networks

- Why might we prefer a deep model over a wide and shallow model for large data sets? (For small / moderate data people still uses FFNN, RBFN, and BNN due to their interpretability and accuracy)



Deep Network



Shallow Network

- An informal justifications:
- Deep models can reuse computational subroutines (and are more compact)
- Learning Certain functions may require a huge number of units in a shallow model.

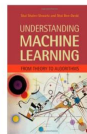
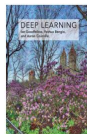
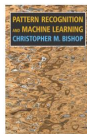
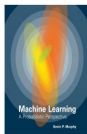
NO FREE LUNCH !

- Able to learn any complex nonlinear mapping or approximate any continuous function.
- As a nonparametric model, it doesn't make any prior assumption about the data distribution or input-output mapping function.
- ANN are very flexible with respect to incomplete, missing and noisy data. ANN are "fault tolerant".
- Neural network when applied to limited data can overfit the training data and lose generalization capability.
- ANNs can't deal with sequential or "temporal" data.
- The selection of the network topology and its parameter lack theoretical background, it is often a "trial and error" matter.
- ANNs lack memory. Solution: Recurrent Neural Networks.

Start with a book of 150 Pages



Then you can start reading these books



- ① Free Datasets: [Link](#)
- ② Free Resources: [Link](#)
- ③ Stanford's CS-229: [Link](#)
- ④ CA Murthy's Lectures: [Link](#)
- ⑤ Andrew Ng's Lectures: [Link](#)
- ⑥ Patrick Winston's Lectures: [Link](#)
- ⑦ Probabilistic ML Course: [Link](#)
- ⑧ Sebastian Raschka's Blog: [Link](#)
- ⑨ Radford M Neal's Blog: [Link](#)
- ⑩ Tanujit Chakraborty's Blog: [Link](#)

Thank You